

Java Collections Framework Handbook (Master Notes)

=====

What this handbook is all about:

This PDF is a Java Collections Framework handbook designed mainly for learning and interview preparation. It covers List, Set, Queue, Deque, Map, their implementations, internal working, commonly used methods, time complexities, and practical applications such as BFS and DFS.

Core Framework Hierarchy:

1. Iterable Interface -> Collection Interface

- List Interface: Ordered collection allowing duplicates (ArrayList, LinkedList, Vector, Stack).
- Set Interface: Unordered collection prohibiting duplicates (HashSet, LinkedHashSet, TreeSet).
- Queue & Deque Interface: FIFO / double-ended queues for graph algorithms (PriorityQueue, ArrayDeque, Link

2. Map Interface (Separate Hierarchy): Stores key-value mappings (HashMap, LinkedHashMap, TreeMap, Concur

Practical Graph & Interview Applications:

- Breadth-First Search (BFS): Uses Queue / ArrayDeque for level-order traversal.
- Depth-First Search (DFS): Uses Deque / ArrayDeque as a lifo stack for path exploration.

Chapter 1: ArrayList Deep Dive

=====

1. What is ArrayList?

ArrayList is a resizable, dynamic array implementation of the List interface in Java.

Unlike standard arrays with fixed size, an ArrayList automatically grows as elements are added.

2. Internal Working and Resizing:

- Default initial capacity: 10 elements.
- Growth factor: When capacity is exceeded, ArrayList creates a new array of size $(oldCapacity + (oldCapacity >> 1))$ which is approximately 1.5x the previous capacity, and copies elements using `System.arraycopy()`.

3. Time Complexity:

- Access / Retrieval (`get`, `set`): $O(1)$ constant time, because it uses index-based direct memory access.
- Insertion at end (amortized): $O(1)$ amortized constant time.
- Insertion / Deletion at arbitrary index: $O(n)$ linear time because following elements must be shifted.
- Search by value (`contains`, `indexOf`): $O(n)$ linear scan.

4. Key Characteristics:

- Maintains insertion order.
- Allows null elements and duplicate values.
- Not synchronized (use `Collections.synchronizedList` or `CopyOnWriteArrayList` for thread safety).
- Uses fail-fast iterators that throw `ConcurrentModificationException` on concurrent structural modification.

Chapter 2: HashMap Deep Dive

=====

1. What is HashMap?

HashMap is a hash table-based implementation of the Map interface, storing data in key-value pairs (K, V). It provides fast lookups, insertions, and deletions based on the hashCode() and equals() methods of the keys.

2. Internal Architecture:

- Array of Nodes (Buckets): Internally represented as Node<K,V>[] table.
- Default Initial Capacity: 16 buckets.
- Default Load Factor: 0.75 (threshold = capacity * loadFactor = 12 elements before resizing/rehashing).
- Index calculation: $\text{index} = (n - 1) \& \text{hash}(\text{key})$, where n is array length (power of two).

3. Collision Handling & Treeification (Java 8+):

- Chaining: When two keys produce the same bucket index, entries are stored in a linked list.
- Treeification Threshold: When a single bucket's linked list length reaches 8 and the table capacity is at least 64, the bucket is converted from a singly-linked list into a balanced Red-Black Tree (TreeNode).
- This improves worst-case lookup from $O(n)$ down to $O(\log n)$.

4. Time Complexity:

- put(key, value) and get(key): $O(1)$ average constant time.
- Worst-case (severe hash collisions): $O(\log n)$ due to Red-Black Tree optimization.
- Allows one null key and multiple null values.

Chapter 3: Set, Queue, and Time Complexity Summary

=====

1. Set Implementations:

- HashSet: Backed by an internal HashMap. $O(1)$ operations. No ordering.
- LinkedHashSet: Maintains insertion order via a doubly-linked list. $O(1)$ operations.
- TreeSet: Backed by a Red-Black Tree (NavigableMap). Maintains sorted natural order. $O(\log n)$ operations.

2. Queue & Deque Implementations:

- PriorityQueue: Backed by a min-heap. Provides $O(\log n)$ insertion and removal of the highest priority element.
- ArrayDeque: Resizable circular array implementation of Deque. $O(1)$ amortized insertion/removal at both ends. Faster than LinkedList and Stack for BFS traversal and LIFO stack operations.

3. Quick Time Complexity Reference Table:

Data Structure	Access	Search	Insertion	Deletion
-----+-----+-----+-----+-----				
ArrayList	$O(1)$	$O(n)$	$O(1)^*$	$O(n)$
LinkedList	$O(n)$	$O(n)$	$O(1)$	$O(1)$
HashMap	N/A	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
HashSet	N/A	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
TreeSet / TreeMap	N/A	$O(\log n)$	$O(\log n)$	$O(\log n)$
ArrayDeque	$O(1)$ ends	$O(n)$	$O(1)$ ends	$O(1)$ ends
PriorityQueue	$O(1)$ min	$O(n)$	$O(\log n)$	$O(\log n)$